

Software Architecture and Design Specification

Vehicle Parking System

Project: Vehicle Parking System
Version: 1.0
Authors: Rohan Chirbi - PES2UG24CS408,
Rohith S - PES2UG24CS409,
Ronak Thamaran - PES2UG24CS410,
Aditya Singh - PES2UG24CS412
Date: 01-10-2026
Status: Draft for Submission

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Team 10	Initial Vehicle Parking System SAD

Approvals

Role	Name	Signature Date
Course Coordinator		
Project Team	Team 10	

1 Introduction

1.1 Purpose

This document defines the architecture and detailed design for the Vehicle Parking System (VPS). It translates the approved SRS requirements and the associated test plan into components, interfaces, data handling, security controls, interaction flows, and operational design decisions that can guide implementation and verification.

1.2 Scope

The design covers a responsive web application for registration and login, parking-slot discovery and reservation, manual or simulated vehicle entry and exit, occupancy tracking, fee calculation, simulated payment recording, receipt generation, and administration. The base system does not integrate physical barriers, ANPR cameras, parking sensors, or a real external payment gateway.

1.3 Audience

The intended readers are the development team, QA testers, project evaluators, course instructors, administrators, and future maintainers of the Vehicle Parking System.

1.4 Definitions

Term	Meaning
VPS	Vehicle Parking System

JWT	JSON Web Token used for authenticated sessions
RBAC	Role-based access control for User and Admin operations
ANPR	Automatic Number Plate Recognition
CRUD	Create, Read, Update, and Delete
RTM	Requirements Traceability Matrix

2 Document Overview

2.1 How to Use This Document

Section 3 defines the system architecture, responsibilities, technology choices, risks, requirement mappings, and security design. Section 4 defines implementation-level behavior through interaction diagrams, API contracts, error handling, monitoring, and UX guidance. Appendix material provides terminology and the documents used as design inputs.

2.2 Related Documents

Document	Version Date	Use in this SAD
Software Requirements Specification	1.0 03-09-2026	Authoritative functional, non-functional, and security requirements
Software Test Plan	1.0 17-09-2026	Verification approach, test cases, and requirement traceability

3 Architecture

3.1 Goals and Constraints

Area	Design Goal or Constraint
Functional coverage	Support all SRS functions: authentication, slots, reservations, entry and exit, billing, payment, administration, and reports.
Security	Protect credentials with salted one-way hashes, enforce JWT-based access and roles, validate input, use HTTPS/TLS, and lock an account after the defined failed-login threshold.
Performance	Support the SRS target of 90th-percentile page/API responses of no more than 2 seconds under normal load and stable operation for at least 50 concurrent users.
Implementation	Use Python with Flask, JSON REST interfaces, and SQLite within an academic-semester scope.
Scope boundary	Treat entry and exit as UI-driven or manually recorded events; do not depend on real barriers, cameras, sensors, or an external payment gateway.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns	Architecture Response
Vehicle Owner	Simple booking, accurate availability and fees, secure account access	Responsive UI, current slot status, reservation validation, fee display before confirmation
Parking Admin Operator	Fast operational updates, rates, visibility of occupancy and revenue	Role-protected administration, tracking records, configurable rates, reports
QA Team	Repeatable verification and observable outcomes	REST boundaries, audit-friendly records, requirement-to-module traceability
Project Evaluator	Scope control, maintainable structure, security evidence	Layered Flask design, documented decisions, measured NFR controls

3.3 Component UML Diagram

Vehicle Parking System Component Architecture

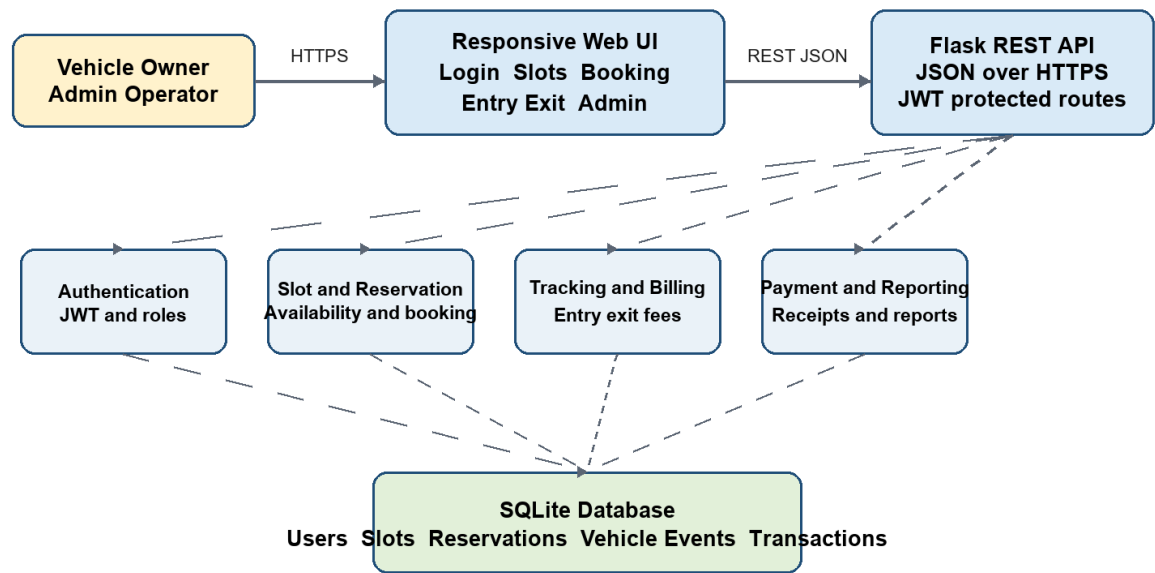


Figure 1 Vehicle Parking System component architecture

3.4 Component Descriptions

Component	Responsibilities	Key Dependencies
Responsive Web UI	Provides registration, login, availability, reservation, entry/exit, payment, and admin views for desktop and mobile screens.	Flask REST API over HTTPS
Flask REST API	Accepts JSON requests, applies authentication and authorization, validates input, and exposes application operations.	JWT helper, service modules, SQLite
Authentication Module	Registers users, verifies password hashes, issues or invalidates JWT sessions, and enforces failed-login lockout.	User records and password hashing library
Slot and Reservation Module	Displays availability, filters by vehicle type, prevents duplicate booking, and creates time-window reservations.	Slot and reservation records
Tracking and Billing Module	Records entry and exit events, updates occupancy, calculates duration-based fees using configured rates.	Vehicle events, rate data, slot status
Payment and Reporting Module	Records simulated payments, generates receipt data, and aggregates occupancy and revenue reports.	Transactions and vehicle-event records
SQLite Database	Persists users, slots, reservations, vehicle events, rates, payment transactions, and audit data.	Flask data-access layer

3.5 Chosen Architecture Pattern and Rationale

The VPS uses a layered monolithic web architecture: a responsive presentation layer calls a Flask REST API; application modules implement the business rules; and a SQLite persistence layer stores system records. This structure gives the project clear separation between UI behavior, authorization, business rules, and storage while retaining a deployment model suitable for the academic scope. A microservice design is not selected because it would add operational and integration complexity without serving a stated requirement.

3.6 Technology Stack and Data Stores

Layer	Technology	Design Use
-------	------------	------------

Client	Responsive HTML CSS JavaScript	Browser UI for User and Admin workflows
Server	Python and Flask	REST API, request handling, validation, and business services
Persistence	SQLite via Python sqlite3	Transactional storage for system entities
Interchange	JSON	Client-server request and response payloads
Session security	JWT	Authenticated session token for protected routes
Transport security	HTTPS TLS	Protected client-server communication
Payment	Simulated internal workflow	Records a transaction and produces receipt data without an external gateway

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Concurrent reservation attempt	Two users could target one slot	Validate availability inside the reservation operation and reject occupied or already-reserved slots with a conflict result.
Incorrect fee due to inconsistent time or rate data	Incorrect payment amount or receipt	Persist entry and exit timestamps, retrieve the configured vehicle-type rate, and verify calculations through TC-Bill-01 and TC-Bill-02.
Unauthorized admin operation	Data or configuration could be changed by a user	Apply JWT authentication and role checks to all admin routes; verify using TC-Sec-02.
Credential compromise	User accounts could be exposed	Store only salted one-way password hashes, use HTTPS/TLS, validate inputs, and enforce lockout after failed-login attempts.
SQLite scaling limit	Load target may be missed	Use indexed lookups for common slot, reservation, and user queries; test the required 50 concurrent sessions and monitor response percentiles.

3.8 Traceability to Requirements

Requirement Group	Design Components	Test Coverage
PVT-F-001 to PVT-F-003	Responsive Web UI, Flask REST API, Authentication Module	TC-Auth-01 to TC-Auth-03
PVT-F-004 to PVT-F-007	Slot and Reservation Module, SQLite	TC-Slot-01 to TC-Slot-04
PVT-F-008 to PVT-F-011	Tracking and Billing Module, SQLite	TC-Track-01 to TC-Track-04
PVT-F-012 to PVT-F-014	Tracking and Billing Module, Payment and Reporting Module	TC-Bill-01, TC-Bill-02, TC-Pay-01
PVT-F-015 to PVT-F-017	Admin UI, Payment and Reporting Module	TC-Admin-01 to TC-Admin-03
PVT-NF-001 to PVT-NF-005	All layers with performance, responsive UI, and credential controls	TC-Perf-01, TC-Perf-02, TC-UX-01, TC-Sec-01
PRJ-SR-001 to PRJ-SR-005	Authentication Module, Flask API security middleware, UI validation	TC-Sec-01 to TC-Sec-05

3.9 Security Architecture

Security is enforced at the browser-to-API boundary and within protected application operations. HTTPS/TLS protects communications. The authentication module stores passwords only as salted one-way hashes and issues JWT sessions after successful login. Flask route guards validate the token and role before protected user and administrator actions proceed. Input validation is applied before persistence or business-rule processing, and error responses avoid exposing implementation details.

Threat	Control	Verification
Spoofing	JWT session validation, secure login flow, password-hash verification	TC-Auth-02 and TC-Sec-01

Tampering	Role checks, input validation, server-side fee and status calculations	TC-Sec-02 and TC-Sec-04
Information disclosure	HTTPS/TLS and no plaintext password storage	TC-Sec-01 and TC-Sec-03
Denial of service through repeated login	Failed-login counter and 15-minute lockout on the sixth failed attempt	TC-Sec-05
Elevation of privilege	Separate User and Admin authorization on API operations	TC-Sec-02

4 Design

4.1 Design Overview

A browser action is transformed into a JSON request by the responsive web UI. The Flask API authenticates and authorizes the request where required, validates the payload, delegates the business operation to the appropriate module, persists the resulting state in SQLite, and returns a JSON success or error response. Business rules such as slot conflict prevention, fee calculation, and role restrictions remain server-side so UI behavior cannot bypass them.

4.2 UML Sequence Diagrams

Sequence Diagram 1 Reserve a Parking Slot

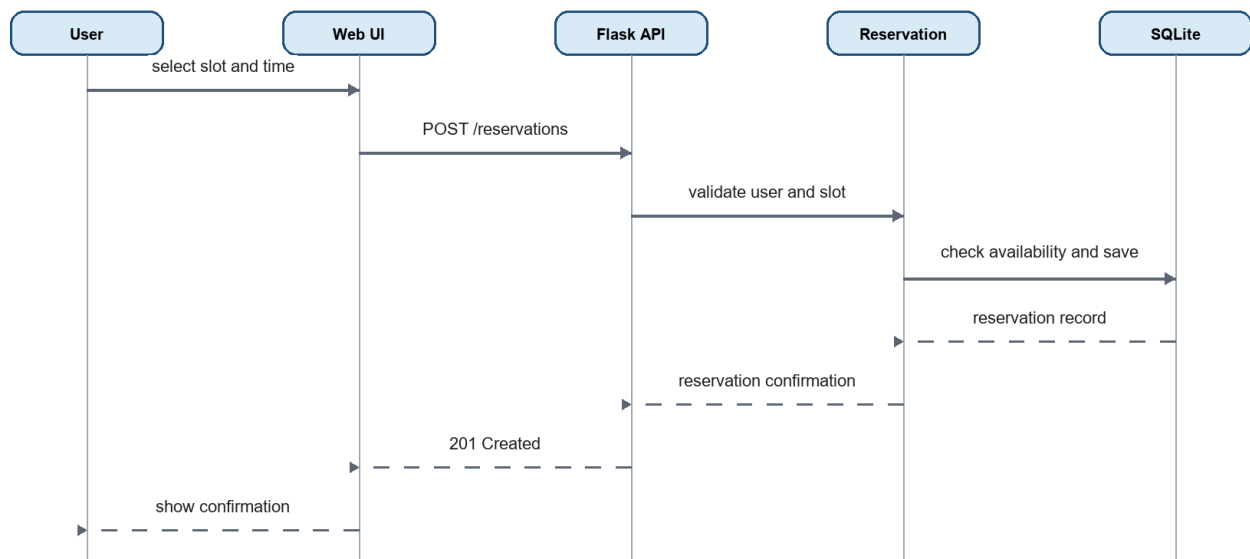


Figure 2 Reservation flow

The reservation flow requires an authenticated user and an available slot. The server validates the selected time window and current slot state before creating a reservation. A conflicting reservation or occupied slot returns an error rather than changing the slot state.

Sequence Diagram 2 Record Exit and Simulated Payment

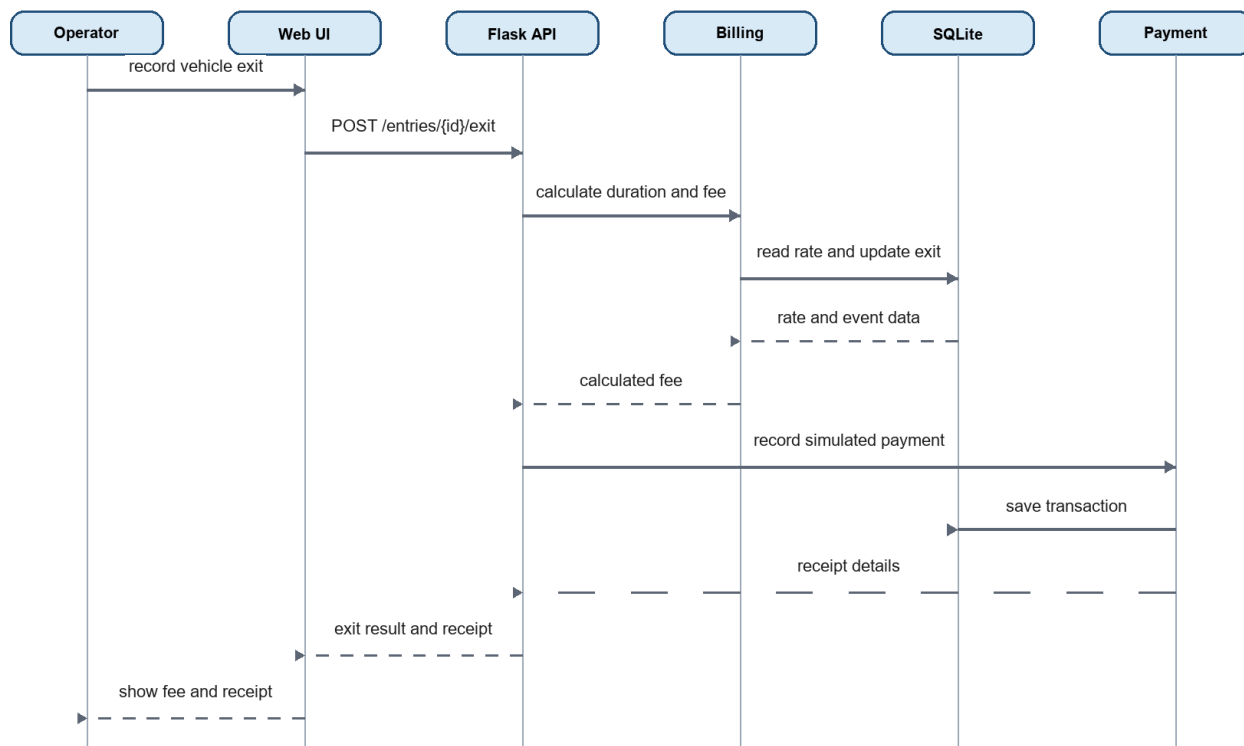


Figure 3 Vehicle exit, fee calculation, and simulated payment flow

The exit flow records the exit timestamp, calculates the fee from duration and configured vehicle-type rate, records a simulated payment transaction, and returns receipt details. Slot availability is released only as part of the completed exit operation.

4.3 API Design

The following REST endpoints are the proposed component interfaces. They use JSON payloads and HTTPS/TLS in deployment. Exact response field names may be refined during implementation, but authorization and business rules remain mandatory.

Endpoint	Method	Purpose	Authorization and Result
/auth/register	POST	Create a user account from name, email, and password.	Public. Success creates account; invalid input returns validation error.
/auth/login	POST	Authenticate email and password and issue session token.	Public. Success returns JWT; invalid credentials or lockout returns error.
/slots	GET	List current slot availability and allow vehicle-type or zone filters.	Public or authenticated per implementation. Returns matching slots and states.
/reservations	POST	Reserve a free slot for a specified time window.	Authenticated User. Returns confirmation or conflict for reserved or occupied slot.
/entries	POST	Record vehicle number, slot ID, and entry timestamp.	Authorized operation. Creates event and marks slot occupied.
/entries/{entryId}/exit	POST	Record exit, calculate fee, and release the slot.	Authorized operation. Returns duration, fee, and exit result.
/payments	POST	Record a simulated payment for a calculated fee.	Authenticated User or authorized operator. Returns transaction and receipt details.
/admin/slots and /admin/rates	POST PUT DELETE	Manage slots and vehicle-type hourly rates.	Admin only. Returns changed configuration or authorization error.

/admin/reports	GET	Generate occupancy and revenue summaries.	Admin only. Returns daily or monthly report data.
----------------	-----	---	---

4.4 Error Handling Logging and Monitoring

Concern	Design
API errors	Return consistent JSON errors with HTTP status codes. Use 400 for invalid input, 401 for unauthenticated access, 403 for forbidden roles, 404 for missing resources, and 409 for reservation conflicts. Do not disclose passwords, tokens, database details, or stack traces.
Audit records	Record entry and exit events with timestamps. Retain payment transaction ID, amount, and timestamp so receipts and reports can be verified.
Application logging	Log request failures, authorization denials, reservation conflicts, exceptions, and performance measurements without sensitive credentials or full JWT values.
Monitoring	Track 90th-percentile page/API response time, concurrent-session behavior, failed-login lockouts, database errors, and operational uptime. Use these records for PVT-NF-001 through PVT-NF-003 verification.

4.5 UX Design

- Use responsive layouts that remain usable on desktop and mobile screen sizes, as required by PVT-NF-004.
- Show slot state clearly as Available, Reserved, or Occupied and show vehicle-type and zone filters near the result list.
- Confirm reservation and payment actions with concise status messages and show fee information before exit confirmation.
- Present validation and authorization errors in clear language without revealing internal security details.
- Keep admin operations separate from general user flows and visibly identify actions that change slots, rates, or reports.

4.6 Open Issues and Next Steps

Item	Decision Needed or Next Step
Fee rounding	Confirm whether partial parking hours are rounded up, rounded to a fixed interval, or billed exactly by duration before implementation of the billing rule.
Reservation expiry	Define the timeout and release behavior for an unused reservation so the availability state remains accurate.
Role permissions	Confirm whether the Parking Lot Admin and Operator are separate roles or share the Admin permissions defined in the SRS.
Operational monitoring	Select the uptime and response-time measurement method to produce the PVT-NF-003 operational report.
Future hardware integration	If cameras, sensors, or barriers are later introduced, define an adapter interface without changing the existing manual entry/exit workflow.

5 Appendices

5.1 Glossary

Term	Definition
Availability	A slot state indicating that the slot can be reserved or assigned.
Occupancy	The count or state of slots currently occupied by vehicles.
Receipt	A generated record containing the simulated payment transaction ID, amount, and timestamp.
Reservation	A booking of an available slot for a specified time window.
Salted hash	A one-way password representation combined with a unique salt so plaintext passwords are not stored.

5.2 References

- Vehicle Parking System Software Requirements Specification, Version 1.0, 03-09-2026.
- Vehicle Parking System Software Test Plan, Version 1.0, 17-09-2026.
- Flask documentation for REST API implementation.
- SQLite documentation for embedded relational data storage.
- OWASP guidance for authentication, access control, input validation, and secure error handling.

5.3 Tools

Proposed implementation and verification tools include Python, Flask, SQLite, browser developer tools, API testing tools, browser-based responsive testing, and an HTTP load-testing tool capable of generating the required concurrent sessions.